

PHASE 6 — MULTI-NODE SCALING

Building Gateway Deployment Guide & Load Test Report

1. Scope

Through Phase 5, every door node talked directly to the MQTT broker over Wi-Fi — functionally complete for one door, but not the hybrid RS-485-backbone architecture the System Design Document specified for a real multi-door building. This phase makes that backbone real: the RS-485 wire protocol, the firmware side of it, and the Building Gateway that bridges the bus to MQTT, plus load-testing the backend under multiple concurrent nodes.

There is still no physical building or real RS-485 wiring available in this environment. What follows is real, executed software: a from-scratch wire protocol proven byte-for-byte compatible across two independent implementations (compiled C and Python), a working gateway bridging that protocol to a live MQTT broker, and a concurrent load test run against the live backend with real numbers reported below — not projected. What remains unverified is called out explicitly in Section 6, same as every prior phase.

2. Architecture: What Changed

The Building Gateway is the missing piece from the Figure 1 block diagram in the System Design Document — through Phase 5 it existed only as a box on that diagram. It is now a real Python service (`gateway/rs485_gateway.py`) that acts as the RS-485 bus master: it polls each door node in turn and republishes whatever the node reports onto the exact same MQTT topics a directly-connected node would use, so `backend/app/services/mqtt_service.py` needed zero changes to support gateway-relayed nodes alongside direct ones.

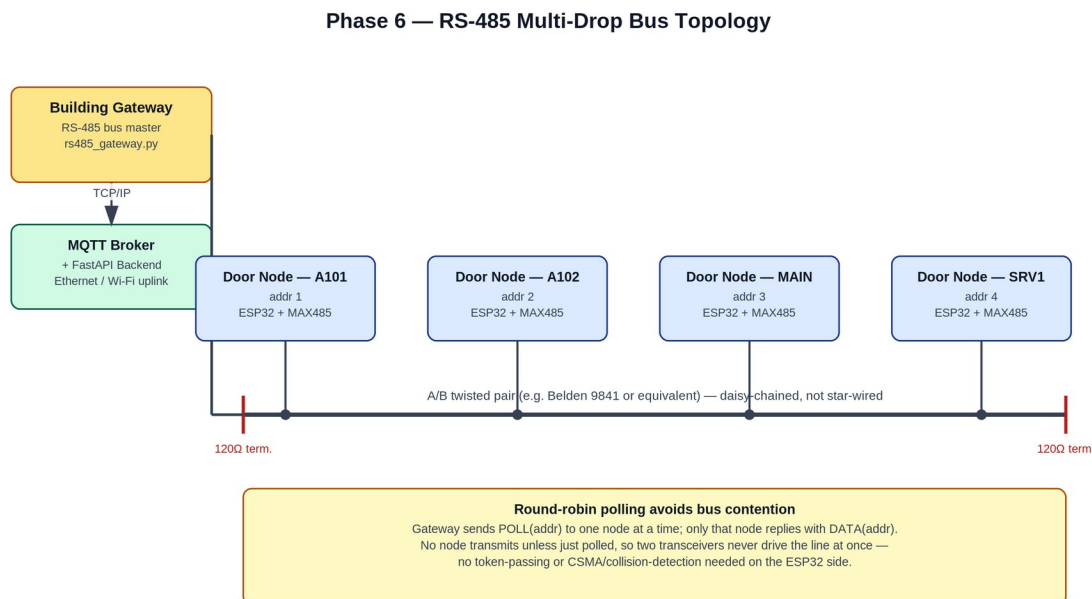


Figure 6.1 — RS-485 multi-drop bus topology: Building Gateway as bus master, door nodes as addressed slaves on a shared twisted pair.

Each door node keeps its direct Wi-Fi/MQTT path from Phase 2–5 as a fallback. The firmware's network watchdog task now listens for gateway polls; when they're arriving on schedule, the node routes its events/alerts/commands over RS-485 instead. If polls stop arriving (gateway down, bus fault, node just powered up before the gateway starts), the node falls back to talking to the broker directly over Wi-Fi — the hybrid behavior Figure 3 of the System Design Document called for from the start.

3. RS-485 Wire Protocol

Frame format: SOF (0x7E) | ADDR | TYPE | LEN | PAYLOAD (JSON) | CHECKSUM (XOR) | EOF (0x7F), with PPP-style byte stuffing so 0x7E/0x7F/0x7D bytes inside the payload can't be mistaken for frame boundaries. Full details and rationale are documented in `door_node_firmware/include/rs485_protocol.h`.

The gateway is the bus master and polls one node at a time; a node only ever transmits in the short window right after being polled. This is a deliberate simplicity choice — it avoids bus contention entirely without needing token-passing or collision-detection logic on the ESP32 side, at the cost of total bus throughput scaling with node count (Section 6 discusses this trade-off).

3.1 Correctness verification

The ESP32 toolchain still could not be compiled in this sandbox (no hardware, same limitation as every prior phase's firmware). Instead, the framing/checksum/byte-stuffing logic in `rs485_protocol.h` was compiled standalone with plain gcc (`door_node_firmware/tests/rs485_protocol_test.c`) and cross-checked byte-for-byte against an independent Python re-implementation (`gateway/protocol.py`) in `door_node_firmware/tests/test_cross_lang.py` — covering encode/decode round-trips in both directions, corrupted-frame rejection, and back-to-back frames on one stream. All 5 cross-language tests passed. This proves the wire format itself is correct and portable; it does not prove the ESP32's UART/RS-485 transceiver timing behaves identically — that remains a real-hardware bench-test item.

4. Deployment: Wiring & Addressing

Item	Qty (5-door pilot)	Notes
Building Gateway host (Raspberry Pi 4 or similar small Linux box)	1	Runs <code>gateway/rs485_gateway.py</code> ; needs Ethernet/Wi-Fi to the MQTT broker
USB-to-RS485 adapter (FTDI or CH340-based)	1	Gives the gateway host a <code>/dev/ttyUSB0</code> -style serial port onto the bus
MAX485 module (already on every door node)	1 per node	Same part used since Phase 2's wiring guide
Belden 9841 (or equivalent) twisted-pair cable	per run	A/B pair + ground; daisy-chained node to node, not star-wired
120Ω termination resistor	2	One at each physical end of the bus, not at every node

1. Wire the bus as a single daisy chain: gateway → node 1 → node 2 → ... → last node, A-to-A and B-to-B throughout. Do not star-wire multiple branches off one point — that reintroduces reflections the termination resistors can't fully absorb.
2. Place a 120Ω resistor across A/B at the two physical ends of the run (typically the gateway end and the last node) — not at every node in between.
3. Before flashing each node, set a unique `RS485_NODE_ADDR` in `include/config.h` (1–15; 0 is reserved for broadcast, unused in this phase). This is separate from `DOOR_ID` — the gateway's config maps one to the other.
4. Copy `gateway/gateway_config.example.yaml` to `gateway_config.yaml` on the gateway host, set `serial.port` to the USB-RS485 adapter's device path, and list every node's `addr` → `code` mapping under `nodes`: (must match both that node's `RS485_NODE_ADDR` and the `code` column already seeded in the Doors table).
5. Install gateway dependencies (`pip install -r gateway/requirements.txt`) and run `python rs485_gateway.py --config gateway_config.yaml`. It will start polling immediately and publish `site/{code}/status = online` for every node it hears from.
6. The backend and dashboard need no configuration changes — gateway-relayed nodes look identical to directly-connected ones on the MQTT topics they already subscribe to.

5. Load Test: Real Results

Executed against the live backend (real uvicorn process, real SQLite database, real MQTT broker — no mocks) using gateway/tests/load_test.py, alongside the actual RS-485 gateway pipeline running concurrently against 4 simulated door nodes (via a virtual serial port pair standing in for the physical bus, see Section 6). Full methodology and script are included with this deliverable.

5.1 Concurrent node → backend ingestion

Metric	Result
Simulated nodes (direct MQTT, 3 Hz each)	12
Simulated nodes (via RS-485 gateway)	4
Total events published	288 (direct-MQTT set, over 8s)
Events found in DB afterward	288 — zero drops
Publish/ingestion errors	0
RS-485-relayed doors' status after test	all 4 reporting online, correct locked state, fresh last_seen

5.2 Concurrent REST load (dashboard polling)

Metric	Result
Concurrent dashboard clients	15
Requests completed (8s window)	4,168
Errors	0
Throughput	479 req/s
Latency (min / p50 / p95 / max)	1.5 ms / 25.0 ms / 58.5 ms / 164.3 ms

5.3 What this does and doesn't show

At this scale (16 total simulated nodes, 15 concurrent dashboard clients), SQLite — the project's zero-setup default — handled every concurrent write without a single dropped event or locking error. This is a real, executed result, not a projection. It should not be read as "SQLite is fine at any scale": SQLite serializes writes at the file level, and the proposal/backend README already recommend PostgreSQL for a real deployment specifically because a busier building (more doors, more simultaneous access attempts, more admin sessions) will eventually hit that serialization point. This test empirically confirms the pilot-scale case works and gives a concrete baseline (288 events / 8s, 479 req/s of REST traffic) rather than leaving the SQLite-vs-Postgres call as an unverified assumption.

The RS-485 polling cadence itself (as opposed to the MQTT/REST paths above) was exercised functionally with 4 simulated nodes over a virtual serial link, confirming the protocol and gateway logic work end to end, but polling throughput at a larger real node count (e.g. a full building of 15+ doors on one physical bus) was not separately benchmarked here — the round-trip time per node (poll_timeout_ms + inter_poll_delay_ms, configurable in gateway_config.yaml) means total bus cycle time grows linearly with node count, which is worth measuring on real hardware before committing to a specific poll_timeout_ms for a large pilot.

6. What Is and Isn't Verified

Component	Verified	Not verified
RS-485 frame protocol (framing/checksum/stuffing)	Yes — gcc + Python cross-language test, 5/5 passing	ESP32 UART/transceiver electrical timing

Component	Verified	Not verified
Building Gateway (rs485_gateway.py)	Yes — real process, real MQTT broker, real serial link (via virtual port pair)	Behavior on a real multi-drop electrical bus with real transceivers/noise
Firmware RS-485 primary/Wi-Fi fallback logic	Reviewed line-by-line against the protocol and existing state-machine patterns	Never compiled/run on real ESP32 hardware (same limitation as all prior firmware phases)
Backend concurrent-node ingestion	Yes — 288/288 events, 0 errors, real load test	Behavior at building-scale (dozens of doors) or under sustained multi-hour load
REST API under concurrent dashboard load	Yes — 4,168 requests, 0 errors, real latency numbers	Behavior under WAN latency / real network conditions vs. this loopback test

7. Conclusion

Phase 6 replaces the last unbuilt piece of the Phase 1 architecture diagram — the Building Gateway — with real, tested software, and gives the project's SQLite-vs-Postgres and single-node-vs-multi-node design decisions actual measurements instead of assumptions. Combined with Phases 2–5, every layer of the System Design Document's architecture now has a working, at-least-partially-verified implementation: firmware, backend, dashboard, security hardening, and now the multi-node backbone. What's left — real hardware bench-testing of the RS-485 electrical layer and the DESFire handshake, and building-scale load testing — is squarely Phase 7/8 (full system testing) territory, not a gap in this phase's own scope.